Slayt 1: Giriş ve Hoş Geldiniz

Görsel/Tasarım Önerisi: Modüler yapıyı simgeleyen renkli Python yapboz parçaları ve 10:00 - 11:30 zaman dilimi ikonu.

• Bugün kod yazma üslubumuzu tamamen değiştiriyoruz! Tekrarlayan spagetti kodlara son veriyoruz.

• Yazılım geliştirme sürecinde kodlarımızı modüler parçalara bölerek yönetilebilir ve ölçeklenebilir mimariler kurgulayacağız.

• Ders boyunca aktif olarak kendi IDE'nizde kod yazacak ve anlık mini pratikler yapacaksınız.

```
# Bugün öğreneceğimiz temel vizyon:
def moduler_program():
    return "Temiz ve Sürdürülebilir Kod!"

print(moduler_program())
```

Sıra Sende (Mini Uygulama): IDE'nizi (VS Code / PyCharm) açın ve bugün için `gun5_fonksiyonlar.py` adında temiz bir Python dosyası oluşturun.

Slayt 2: Gündem ve Öğrenme Hedefleri

Görsel/Tasarım Önerisi: 4 aşamalı hedef tahtası simgesi ve %0'dan %100'e ilerleyen öğrenme çubuğu.

- Tekrarlayan kod bloklarını temizleme ve DRY (Don't Repeat Yourself) prensibini benimseme.
- `def` anahtar kelimesi ile sıfırdan fonksiyon tanımlama ve çalıştırma.
- Parametre alan ve dışarıdan esnek veri kabullenen alt programlar tasarlama.
- `return` mekanizması ile değer döndüren modüler mimariler inşa etme.

```
# Günün ana hedef özeti:
def hedef_kontrol(tamamlandi=True):
    if tamamlandi:
        print("Kazanımlar Eksiksiz Alındı!")

hedef_kontrol()
```

Sıra Sende (Mini Uygulama): Oluşturduğunuz Python dosyasının en üstüne yorum satırlarıyla (`#`) bugünkü hedeflerinizi kendi kelimelerinizle yazın.

Slayt 3: Problem: Tekrarlayan Kod Kirliliği

Görsel/Tasarım Önerisi: Karışık spagetti kablolar vs son derece düzenli sunucu kablo düzeni karşılaştırması.

• Aynı kod bloğunu programın farklı yerlerinde kopyala-yapıştır yapmak en büyük yazılım hatalarından biridir.

• Kodda küçük bir değişiklik gerektiğinde, kopyaladığınız 10 ayrı yeri tek tek güncellemek zorunda kalırsınız.

• Bu durum hata yapma riskinizi artırır ve kod okunabilirliğini yok eder.

```
# KÖTÜ PRATİK (Spagetti Kod):
# Müşteri 1 karşılama
print("Sayın Ahmet Öz, sistemimize hoş geldiniz!")
print("Hesap durumunuz: Aktif")

# Müşteri 2 karşılama
print("Sayın Ayşe Yılmaz, sistemimize hoş geldiniz!")
print("Hesap durumunuz: Aktif")
```

Sıra Sende (Mini Uygulama): Yukarıdaki kötü pratik kod bloğunu inceleyin. İki karşılama işleminde değişen tek verinin ne olduğunu tespit edin.

Slayt 4: Çözüm: DRY (Don't Repeat Yourself) Prensibi

Görsel/Tasarım Önerisi: Copy-Paste üzerine çekilmiş kırmızı engel işareti ve etrafında dönen Reusability (Yeniden Kullanılabilirlik) simgesi.

• Yazılım dünyasının en temel ilkesi: **DRY (Kendini Tekrar Etme!)**

• Bir algoritmayı veya mantığı birden fazla kez yazıyorsanız, onu derhal bağımsız bir fonksiyona dönüştürmelisiniz.

• Böylece kod güncelleneceğinde sadece TEK bir noktayı değiştirmek yeterli olur.

```
# TEMİZ KOD FELSEFESİ:
# "Bir kere yaz, her yerde çalıştır!"
# Tüm tekrarlayan mantık tek bir merkezde toplanır.
# Bakım kolaylığı %100 artar.
```

Sıra Sende (Mini Uygulama): Geçtiğimiz günlerde yazdığınız kodları düşünün. Hangi işlemleri tekrar tekrar kopyaladığınızı yorum olarak not edin.

Slayt 5: Fonksiyon Nedir? (Fabrika Modeli)

Görsel/Tasarım Önerisi: Girdi (Hammadde) -> Fonksiyon (Makine/Fabrika) -> Çıktı (Ürün) huni diyagramı.

- Fonksiyon, belirli bir görevi yerine getirmek için gruplanmış kod bloğudur.
- Onu bir fabrika makinesi gibi düşünebilirsiniz: İstediğiniz zaman çağırırsınız, girdi verirsiniz ve size bir çıktı üretir.
- Programınızı küçük, bağımsız çalışan uzman makinelere bölmenizi sağlar.

```
# Fonksiyonun Fabrika Modeli:
def fabrika_makinesi(girdi):
    # Girdiyi işleme adımı
    islenmis_cikti = girdi * 2
    return islenmis_cikti

sonuc = fabrika_makinesi(5)
print(sonuc) # Çıktı: 10
```

Sıra Sende (Mini Uygulama): Gerçek hayattan bir fonksiyon örneği kurgulayın (Örn: Kahve Makinesi -> Girdi: Su+Kahve, Çıktı: Sıcak Kahve).

Slayt 6: İlk Fonksiyonumuzu Tanımlayalım: `def` Syntax

Görsel/Tasarım Önerisi: `def`, fonksiyon adı, parantez () ve iki nokta (:) elemanlarını vurgulayan renkli kod mimarisi diyagramı.

- Python'da fonksiyon tanımlamak için `def` (define) anahtar kelimesini kullanırız.
- Fonksiyon isminden sonra parantez `()` ve iki nokta `:` koymak zorunludur.
- Fonksiyonun içindeki kodlar mutlaka 4 karakter girintili (indentation) yazılmalıdır.

```
# Fonksiyon Tanımlama (Definition):
def selamla():
    """Bu fonksiyon standart karşılama basar."""
    print("Merhaba! Python Modüler Kod Dünyasına Hoş Geldiniz.")

# Not: Sadece tanımlamak kodu çalıştırmaz!
```

Sıra Sende (Mini Uygulama): `selamla()` adında basit bir fonksiyon tanımlayın. Dosyanızı çalıştırın. Ekrana bir şey basıldı mı? Nedenini düşünün.

Slayt 7: Fonksiyonu Çağırmak (Function Call)

Görsel/Tasarım Önerisi: Düdük ikonu ve fonksiyona doğru tetikleme yapan çağrı okları.

• Bir fonksiyonu tanımlamak onu sadece hafızaya kaydeder; çalıştırmaz.

• Fonksiyon içindeki kodların tetiklenmesi için fonksiyonun adı ve parantezleriyle `selamla()` şeklinde çağrılması gerekir.

• İstediğiniz kadar çağırıp tekrar tekrar kullanabilirsiniz.

```
# Fonksiyon Tanımı:
def selamla():
    print("Sistem Çalışıyor...")

# Fonksiyon Çağrıları (Call):
selamla()  # 1. Çağrı
selamla()  # 2. Çağrı
selamla()  # 3. Çağrı
```

Sıra Sende (Mini Uygulama): Kendi tanımladığınız `selamla()` fonksiyonunu 3 kez üst üste çağırın ve terminal çıktısını gözlemleyin.

Slayt 8: Parametre Alan Fonksiyonlar: Girdi Vermek

Görsel/Tasarım Önerisi: Kutunun içine giren isim etiketi ve kutudan çıkan kişiselleştirilmiş mesaj diyagramı.

- Fonksiyonların dinamik ve esnek olması için dışarıdan veri alabilmesi gerekir.
- Fonksiyon tanımlanırken parantez içine yazılan değişkenlere **parametre** denir.
- Bu parametreler fonksiyon içerisinde normal bir değişken gibi kullanılır.

```
# Parametre alan fonksiyon:
def kisi_selamla(kullanici_adi):
    """Dışarıdan gelen isim parametresini kullanır."""
    print(f"Hoş geldin Sayın {kullanici_adi}! İyi dersler.")

# Argüman geçerek çağırma:
kisi_selamla("Ahmet")
kisi_selamla("Zeynep")
```

Sıra Sende (Mini Uygulama): `kisi_selamla` fonksiyonunu kendi adınızı argüman olarak vererek çalıştırın.

Slayt 9: Çoklu Parametre Kullanımı

Görsel/Tasarım Önerisi: İki farklı girdi musluğu (İsim, Rol) -> Fonksiyon İşleyici -> Ekrana basılan Kullanıcı Kartı.

- Bir fonksiyon birden fazla parametre kabul edebilir.
- Parametreler parantez içinde virgül (`,`) ile birbirinden ayrılır.
- Fonksiyon çağrılırken argümanlar tanımlanan sırayla gönderilmelidir.

```
# Çoklu parametre yapısı:
def kullanici_kaydi(ad, soyad, rol):
    print(f"KAYIT: {ad} {soyad} | Yetki Seviyesi: {rol}")

# Sırasıyla değer gönderme:
kullanici_kaydi("Mehmet", "Kaya", "Yazılımcı")
kullanici_kaydi("Elif", "Demir", "Yönetici")
```

Sıra Sende (Mini Uygulama): Kendi adınız, soyadınız ve 'Öğrenci' rolünüz ile `kullanici_kaydi` fonksiyonunu çağırın.

Slayt 10: Parametre vs Argüman Farkı Nedir?

Görsel/Tasarım Önerisi: Karşılaştırma Tablosu (Parametre = Değişken Tanımı / Argüman = Gerçek Gönderilen Değer).

- Yazılım terminolojisinde bu iki kavram sıkça karıştırılır:
- **Parametre**: Fonksiyon *tanımlanırken* parantez içine yazılan soyut değişken adıdır (`x`, `y`).
- **Argüman**: Fonksiyon *çağrılırken* gönderilen somut gerçek değerdir (`10`, `20`).

```
# Tanımlama anında 'a' ve 'b' PARAMETREDİR:
def topla(a, b):
    print(f"Toplam: {a + b}")

# Çağrılma anında 15 ve 25 ARGÜMANDIR:
topla(15, 25)
```

Sıra Sende (Mini Uygulama): `topla(55, 45)` çağrısındaki parametreleri ve argümanları sözlü olarak ifade edin.

Slayt 11: `print()` ve `return` Arasındaki Hayati Fark!

Görsel/Tasarım Önerisi: Sol tarafta ekrana yazıp veriyi unutan hoparlör (`print`), sağ tarafta veriyi elde tutan kurye (`return`).

- `print()` sadece ekrana geçici bir yazı basar, üretilen veriyi hafızada tutmaz!
- `return` ise fonksiyonun hesapladığı değeri çağrıldığı yere geri teslim eder.
- Böylece fonksiyondan dönen değeri başka işlemlerde, değişkenlerde kullanabilirsiniz.

```
# print() kullanırsa değer kaybolur:
def hatali_topla(a, b):
    print(a + b)

# return kullanırsa değer elde tutulur:
def dogru_topla(a, b):
    return a + b

sonuc = dogru_topla(10, 5)
print(f"Sonucun 2 Katı: {sonuc * 2}") # 30
```

Sıra Sende (Mini Uygulama): `hatali_topla(10, 5) * 2` kodunu çalıştırmayı deneyin. Nasıl bir `TypeError` aldığınızı görün.

Slayt 12: `return` İle Değer Döndüren Fonksiyonlar

Görsel/Tasarım Önerisi: Fonksiyondan çıkıp değişken kabına akan değer akışı simgesi.

- `return` komutu çalıştırıldığı anda fonksiyonun yürütülmesi **DERHAL SONLANIR**.
- `return` satırından sonra yazılan hiçbir kod satırı çalışmaz (Unreachable Code).
- Koşullu yapılarda (`if-else`) erken çıkış yapmak için son derece kullanışlıdır.

```
# return kullanımı ve erken çıkış:
def mutlak_deger(sayi):
    if sayi < 0:
        return -sayi  # Negatifse pozitif yap ve çık
    return sayi       # Zaten pozitifse aynen dön

print(mutlak_deger(-15)) # 15
print(mutlak_deger(20))  # 20
```

Sıra Sende (Mini Uygulama): Bir sayının çift olup olmadığını kontrol edip `True` veya `False` döndüren `cift_mi(sayi)` fonksiyonunu yazın.

Slayt 13: Varsayılan (Default) Parametre Değerleri

Görsel/Tasarım Önerisi: Opsiyonel girdi alanı ve varsayılan değer seçim rozeti.

- Parametrelere eşittir (`=`) operatörü ile varsayılan (default) değer atanabilir.
- Eğer kullanıcı fonksiyonu çağırırken o argümanı vermezse varsayılan değer devreye girer.
- Varsayılan parametreler her zaman parametre listesinin **EN SONUNDA** yer almalıdır.

```
# Default parametre kullanımı:
def indirimli_fiyat(tutar, indirim_orani=0.10):
    """Varsayılan olarak %10 indirim uygular."""
    return tutar * (1 - indirim_orani)

print(indirimli_fiyat(100))      # 90.0 (Default %10 uygulandı)
print(indirimli_fiyat(100, 0.25))# 75.0 (%25 uygulandı)
```

Sıra Sende (Mini Uygulama): `indirimli_fiyat(500)` çağrısı yaptığınızda çıktı ne olur? Kendi IDE'nizde test edin.

Slayt 14: İsimlendirilmiş Argümanlar (Keyword Arguments)

Görsel/Tasarım Önerisi: Hedef etiketli okların doğru parametre kutularına gitmesini gösteren yönlendirme şeması.

- Fonksiyon çağrılırken argümanlar varsayılan olarak pozisyona (sıraya) göre eşleşir.
- Ancak `parametre_adi = deger` şeklinde açıkça isim belirterek sıradan bağımsız çağrı yapabilirsiniz.
- Bu yöntem kodun okunabilirliğini ve anlaşılırlığını muazzam derecede artırır.

```
# Positional vs Keyword Arguments:
def kargo_bilgi(agirlik, mesafe, firma="Yurtiçi"):
    return f"{agirlik}kg ürün {mesafe}km yere {firma} ile taşınıyor."

# Keyword arguments ile sıra bağımsız çağrı:
msg = kargo_bilgi(mesafe=350, agirlik=5)
print(msg)
```

Sıra Sende (Mini Uygulama): `firma='Aras'`, `agirlik=12`, `mesafe=100` keyword argümanlarıyla fonksiyonu çağırın.

Slayt 15: Değişken Kapsamı: Local (Yerel) vs Global Değişkenler

Görsel/Tasarım Önerisi: Evin içi (Local Scope / Fonksiyon İçi) vs Bahçe (Global Scope / Tüm Program) şeması.

- **Local Scope**: Fonksiyon içinde tanımlanan değişkenler sadece o fonksiyon içinde yaşar.
- **Global Scope**: Fonksiyonların dışında, ana programda tanımlanan değişkenlerdir.
- Fonksiyon dışından fonksiyonun içindeki yerel bir değişkene erişemezsiniz!

```
# Scope (Kapsam) Mantığı:
x = 50  # Global değişken

def scope_test():
    x = 10  # Local değişken (Sadece burada geçerli)
    print(f"Fonksiyon içi x: {x}")

scope_test()              # Çıktı: 10
print(f"Fonksiyon dışı x: {x}") # Çıktı: 50
```

Sıra Sende (Mini Uygulama): Fonksiyon içinde `y = 100` değişkeni tanımlayın. Fonksiyon dışında `print(y)` yapmayı deneyin. Ne hatası aldınız?

Slayt 16: Çoklu Değer Döndürme (Multiple Return Values)

Görsel/Tasarım Önerisi: Tek bir hediye kutusundan çıkan 2 ayrı paket (Tuple unpacking) simgesi.

- Python'da bir fonksiyon tek bir `return` satırında virgülle ayırarak birden fazla değer döndürebilir.
- Arka planda bu değerler bir `tuple` olarak paketlenir.
- Dönen değerleri eşzamanlı olarak birden fazla değişkene aktarabilirsiniz (Unpacking).

```
# Çoklu Değer Dönüşü:
def daire_hesapla(yari_cap):
    cevre = 2 * 3.14 * yari_cap
    alan = 3.14 * (yari_cap ** 2)
    return cevre, alan  # İki değer dönüyor

# Unpacking (Ayrıştırma):
c, a = daire_hesapla(5)
print(f"Çevre: {c}, Alan: {a}")
```

Sıra Sende (Mini Uygulama): Yarıçapı 10 olan daire için `daire_hesapla(10)` çağrısı yapıp çevre ve alanı ayrı değişkenlere basın.

Slayt 17: Modüler Yazılım Mimarisi Kurgulama

Görsel/Tasarım Önerisi: Küçük modül bloklarının birleşerek sağlam bir yazılım binası oluşturduğu mimari şema.

- Modüler mimari; büyük ve karmaşık sistemleri küçük, bağımsız çalışan fonksiyonlara bölmektir.
- Her fonksiyon sadece **TEK BİR İŞ** yapmalıdır (Single Responsibility Principle).
- Böylece kodun test edilmesi, okunması ve geliştirilmesi çok kolaylaşır.

```
# Modüler Tasarım Örneği:
def veri_al():
    return float(input("Notunuzu girin: "))

def harf_notu_hesapla(not_degeri):
    return "AA" if not_degeri >= 90 else "FF"

def rapor_yazdir(harf):
    print(f"--- ÖĞRENCİ SONUÇ RAPORU: {harf} ---")
```

Sıra Sende (Mini Uygulama): Bu 3 modüler fonksiyonu sırasıyla çağırarak eksiksiz bir not değerlendirme akışı kurgulayın.

Slayt 18: İnteraktif Uygulama: Modüler ATM Sistemi (Part 1)

Görsel/Tasarım Önerisi: ATM ekranı ve fonksiyon mimarisi akış diyagramı (Bakiye Sorgula / Para Yatır).

- Saat 11:00 - 11:20 canlı uygulama zamanı! Modüler bir ATM simülasyonu inşa ediyoruz.
- İlk aşamada bakiye sorgulama ve para yatırma modüllerini fonksiyonlaştırıyoruz.
- Fonksiyonlarımız mevcut bakiyeyi parametre olarak alıp güncel bakiyeyi `return` edecek.

```
# ATM Modülü - Bölüm 1:
bakiye = 1000.0

def bakiye_sorgula(mevcut):
    print(f"[BİLGİ] Güncel Bakiyeniz: {mevcut} TL")

def para_yatir(mevcut, miktar):
    if miktar > 0:
        yeni = mevcut + miktar
        print(f"[BAŞARILI] {miktar} TL yatırıldı.")
        return yeni
    return mevcut
```

Sıra Sende (Mini Uygulama): `para_yatir(bakiye, 500)` fonksiyonunu çalıştırıp dönen değeri `bakiye` değişkenine atayın.

Slayt 19: İnteraktif Uygulama: Modüler ATM Sistemi (Part 2)

Görsel/Tasarım Önerisi: Para çekme onay mekanizması akış şeması (Bakiye Yeterli mi? Kontrolü).

- Şimdi ATM sistemimize güvenli para çekme modülünü dahil ediyoruz.
- Fonksiyon içinde bakiye kontrolü yaparak yetersiz bakiye durumunda işlemi engelliyoruz.
- Modüler yapımız sayesinde her işlem kendi içinde güvenli ve bağımsız hale geliyor.

```
# ATM Modülü - Bölüm 2:
def para_cek(mevcut, miktar):
    if miktar > mevcut:
        print("[HATA] Yetersiz Bakiye! İşlem engellendi.")
        return mevcut
    else:
        yeni = mevcut - miktar
        print(f"[BAŞARILI] {miktar} TL çekildi.")
        return yeni

# Test:
bakiye = para_cek(bakiye, 400)
bakiye_sorgula(bakiye)
```

Sıra Sende (Mini Uygulama): Bakiyenizden 5000 TL çekmeyi deneyin ve hata kontrol mekanizmasının çalıştığını doğrulayın.

Slayt 20: Sık Yapılan Fonksiyon Hataları ve Debugging

Görsel/Tasarım Önerisi: Kırmızı Böcek (Bug) ikonu ve büyüteç ile hata tespit görseli.

- En sık karşılaşılan 3 fonksiyon hatası:
- 1. `return` yazmayı unutmak (Fonksiyon otomatik olarak `None` döndürür).
- 2. Parametre sayısı ile gönderilen argüman sayısının uyuşmaması (`TypeError`).
- 3. Fonksiyon içinde global değişkenleri habersiz değiştirmeye çalışmak.

```
# HATALI KOD ÖRNEĞİ:
def kare_al(sayi):
    kare = sayi ** 2
    # Hata: return unutuldu!

sonuc = kare_al(4)
print(sonuc) # Ekrana 'None' yazar!
```

Sıra Sende (Mini Uygulama): Yukarıdaki `kare_al` fonksiyonuna `return kare` ekleyerek hatayı düzeltin ve 16 çıktısını elde edin.

Slayt 21: Günün Özet Tablosu ve Mini Quiz

Görsel/Tasarım Önerisi: 4 çeyrekli özet matrisi (Kavram -> Açıklama -> Örnek).

• Özet Kavramlar:

• • `def`: Fonksiyon tanımlar.

• • Parametre: Beklenen girdi | Argüman: Gönderilen gerçek veri.

• • `return`: Hesaplanan değeri dışarı aktarır ve fonksiyonu sonlandırır.

• • Modülerlik: Karmaşık kodu küçük temiz parçalara bölmektir.

```
# HIZLI SORU: Bu kodun çıktısı ne olur?
def test_hesap(x, y=3):
    return (x + y) * 2

print(test_hesap(4)) # Cevap nedir?
```

Sıra Sende (Mini Uygulama): Soru kodunun çıktısını çalıştırmadan tahmin edin, ardından IDE'nizde çalıştırarak doğrulayın.

Slayt 22: Ders Sonu Görevi ve Gelecek Ders Projeksiyonu

Görsel/Tasarım Önerisi: 6 Kasım -> 9 Kasım geçiş okları ve Nesne Yönelimli Programlama (OOP) logosu.

- **Bugünün Ödevi**: 4 işlem (Topla, Çıkar, Çarp, Böl) yapan modüler bir hesap makinesi yazın.
- Her işlemi ayrı bir fonksiyon yapın ve kullanıcıdan seçim alarak çalıştırın.
- **Pazartesi (9 Kasım)**: Nesne Yönelimli Programlamaya (OOP - Class & Object) geçiyoruz!

```
# Gelecek Derse Hazırlık (OOP Ön İzleme):
# Fonksiyonlarımızı ve verilerimizi tek çatı altında
# toplayan Class (Sınıf) yapılarını öğreneceğiz!
class HesapMakinesi:
    pass # Pazartesi görüşmek üzere!
```

Sıra Sende (Mini Uygulama): Hesap makinesi ödev dosyanızı `hesap_makinesi_moduler.py` adıyla kaydedip tamamlayın.